

APR Design Flow

RTL-to-GDS 後端設計流程

從 Floorplan 到 GDS：Cadence Innovus 實作步驟詳解

Agenda

1. 什麼是 APR？在整個晶片設計流程中的位置 + 名詞速查表
2. RTL-to-GDS 九大步驟總覽
3. 兩個對照案例：`gcd`（教學範例） vs `DTMF_CHIP`（180nm 完整實作）
4. 逐步驟詳解（含示意圖）：Design Import → Floorplan → Power Planning → Placement → CTS → Routing → DFM → Verification → Data Export
5. 動手練習：如何用 checkpoint 驗證自己真的理解每個階段
6. 各階段檢查清單總表
7. 進階補充：EDA 工具與授權機制（可跳過）

每個 Step 講完核心流程後，會接一段標示「🔍 進階補充」的深挖內容——時間夠再講，時間不夠可以直接跳過。

資料來源：`note/floorplan.md`、`note/placement.md`、`note/routing.md`、`note/STA.md`、`reference_design/gcd/`、`reference_design/180um/Version4`

什麼是 APR ？

APR (Automatic Place and Route) = 後端物理設計，把合成後的**閘級網表 (gate-level netlist)** 轉換成可以送晶圓廠的版圖 (**GDSII**)。

輸入：

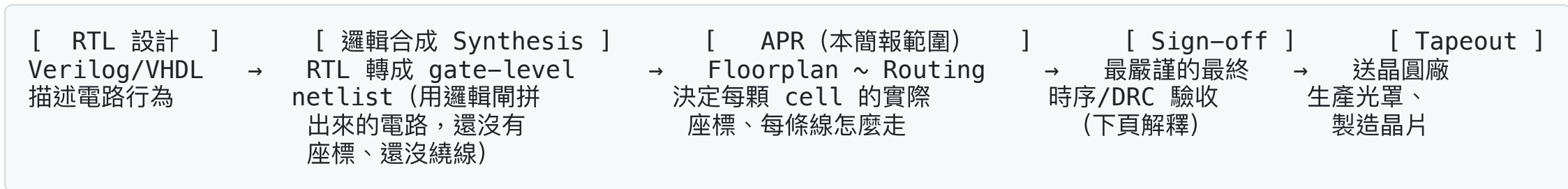
- 合成後網表 (netlist) + SDC 時序約束 + LEF/Lib 製程檔

輸出：

- GDSII 版圖、SDF 延遲檔、post-APR 網表、LEF abstract

核心矛盾：面積、時序、可繞線性 (routability)、功耗 四者互相牽制，整個流程就是反覆疊代收斂這四個指標。

APR 在整個晶片設計流程中的位置



- 合成產出的 **netlist (網表)** 只描述「用哪些邏輯閘、怎麼連接」，還是抽象的邏輯關係
- **APR** 把網表變成實際的 **版圖 (layout / GDSII)**：每顆標準單元放在哪裡、每一條金屬線怎麼繞
- APR 做完要通過 **sign-off** (最嚴謹的驗收) 才能 **tapeout** (送晶圓廠生產) —— 這兩個詞下一頁就會解釋

名詞速查表（後面會一直用到）

名詞	白話解釋
Netlist（網表）	合成後「用哪些邏輯閘、怎麼連接」的電路描述，還沒有座標與繞線
Corner（角落）	製程 / 電壓 / 溫度的一種組合條件（如「慢 + 低壓 + 高溫」），設計要在多個 corner 下都過關
OCV（On-Chip Variation）	考慮同一顆晶片上不同位置也會有微小製程差異的分析方式，比單純 corner 更保守
Sign-off	用最嚴謹的分析標準做的正式驗收，通過才代表這批結果可信賴、能繼續往下走
ECO（Engineering Change Order）	只針對少數幾個 cell 做最小幅度修改解決特定問題，不必重跑整個階段
WNS / TNS	最壞路徑的違規時間 / 所有違規路徑加總，越正（或越接近 0）代表時序越健康
Skew	時脈訊號到達不同暫存器的時間差，CTS 的目標之一就是壓低它
Congestion（壅塞）	某區域要繞的線太多、資源不夠，容易繞不進去或繞很繞
IR Drop / EM	電源網路壓降過大 / 金屬線電流密度過高，兩者都是電源設計要顧慮的可靠度問題
Tapeout	Sign-off 通過後，把最終版圖資料正式送交晶圓廠準備生產

RTL-to-GDS 九大步驟總覽

#	階段	目的
1	Design Import	匯入網表、約束、製程檔
2	Floorplan	決定晶片/核心尺寸、IO 位置
3	Power Planning	建立電源 ring / stripe / rail
4	Placement	擺放巨集與標準單元
5	CTS	合成時脈樹、平衡 skew
6	Routing	訊號線 global + detail routing
7	DFM	Filler cell、dummy metal fill
8	Verification	DRC、LVS、Antenna 檢查
9	Data Export	輸出 GDS、SDF、網表給下游

每一步都伴隨著「做完 → 檢查時序/DRC/壅塞 → 不合格就疊代重做」的迴圈，不是單向線性流程。

兩個對照案例

案例一：gcd（教學範例）

- 路徑：`reference_design/gcd/scripts/gcd_soc.tcl`
- 設計：`gcd`（Greatest Common Divisor，最大公因數電路）— 小型教學電路
- 製程：90nm（`setDesignMode -process 90`）
- 工具：Cadence Innovus（前身 SoC Encounter，檔名 `soc` 即此縮寫）
- 特色：九個步驟各只跑「一次」，關掉時序驅動/SI 驅動 routing，是最簡化的示範流程，方便一次看完全貌

案例二：DTMF_CHIP（180nm 完整實作）

- 路徑：`reference_design/180um/Version4/`（實際 checkpoint 資料庫）
- 設計：`DTMF_CHIP`，180nm 製程（`setDesignMode -process 180`）
- 特色：真實課程作業，每個階段都反覆疊代很多輪才收斂——是本簡報「每階段實際要做什麼」的第一手證據

Step 1 — Design Import

Step 1 : Design Import (設計匯入)

這階段要做什麼：

- 匯入合成後的閘級網表 (netlist)
- 讀入 SDC 時序約束
- 指定 LEF (製程幾何) 與 Lib (時序) 檔案
- 建立 MMMC (Multi-Mode Multi-Corner) 多角多模分析設定
- 讀入 IO 接腳位置約束

對照 `note/floorplan.md` 附錄 Step.1 ; ICC 對應指令為 `create_mw_lib` 、
`import_designs` 、 `read_sdc` 。

Step 1 : Design Import — gcd 範例

```
set TOP_DESIGN "gcd"  
loadConfig ../design_data/${TOP_DESIGN}.conf 1
```

`gcd.conf` 一次集中定義了 netlist、SDC、LEF、IO 檔、MMMC view 檔與 floorplan 預設參數；
`gcd.view` 用 `create_library_set` / `create_analysis_view` 建立 `slow.lib(setup 角)` / `fast.lib(hold 角)` 的 MMMC 設定。

DTMF_CHIP 實際用的完整 MMMC corner / OCV 設定較深入，下面幾頁補充。

Step 1 進階補充（可視時間彈性簡報）

Step 1 補充：DTMF_CHIP 的真實 MMMC 設定

gcd 只用兩個 .lib 檔做示範；reference_design/180um/ 的 checkpoint (viewDefinition.tcl) 還原出一套完整的 corner-based MMMC：

```
create_library_set -name dtmf_libs_min -timing {pllclk_fast.lib ram_128x16A_fast_syn.lib
    rom_512x16A_fast_syn.lib ram_256x16A_fast_syn.lib fast.lib tpz973gbc-lite_fast.lib} -si {fast.cdb}
create_library_set -name dtmf_libs_max -timing {pllclk_slow.lib ram_128x16A_slow_syn.lib
    ram_256x16A_slow_syn.lib rom_512x16A_slow_syn.lib slow.lib tpz973gwc-lite_slow.lib} -si {slow.cdb}

create_rc_corner -name dtmf_rc_corner -cap_table t018s6mlv.capTbl -qx_tech_file t018s6mm.tch \
    -preRoute_res 1 -postRoute_res 1 -preRoute_cap 1 -postRoute_cap 1 -postRoute_xcap 1

create_delay_corner -name dtmf_corner_min -library_set dtmf_libs_min -rc_corner dtmf_rc_corner
create_delay_corner -name dtmf_corner_max -library_set dtmf_libs_max -rc_corner dtmf_rc_corner

create_constraint_mode -name common -sdc_files {dtmf.sdc}
create_analysis_view -name dtmf_view_setup -constraint_mode common -delay_corner dtmf_corner_max
create_analysis_view -name dtmf_view_hold -constraint_mode common -delay_corner dtmf_corner_min
set_analysis_view -setup {dtmf_view_setup} -hold {dtmf_view_hold}
```

Step 1 補充：MMMC 結構拆解

2 組 library set (`_min` = fast 製程角，含 PLL/RAM/ROM/std-cell 全部換成 fast 版； `_max` = slow 版) 交叉 1 個共用 `dtmf_rc_corner` (RC 抽取條件相同，只有 library 角落不同) → 產生 2 個 delay corner → 搭配同一份 `dtmf.sdc` 的 1 個 constraint mode → 組成 2 個 analysis view：

setup 用 slow 角 (`corner_max`)、hold 用 fast 角 (`corner_min`)，符合 setup 抓最壞情況 (慢)、hold 抓最壞情況 (快) 的物理直覺。

Step 1 補充：MMMC 設定一次建立、全程沿用

比對 `Floorplan/01Floorplan_set` 與最終 `Route/05MetalFill` 兩個 checkpoint 的 `viewDefinition.tcl`：**完全相同**（僅多一行 GUI 用的 `set_interactive_constraint_mode`）。
`Version5`（同設計的另一次練習）也是一字不差的同一套 MMC。

意義：MMC 的 corner / view 設定是跟著製程與設計綁定的，在 Design Import 階段建立一次之後，Floorplan → Placement → CTS → Route 全程都重複使用同一套視角，**不會**每個階段重建；不同 checkpoint 之間唯一會變的是「用哪個 view 做檢查」與「有沒有開額外的分析模式」（下一頁）。

Step 1 補充：跑到哪個階段才切換分析模式？

從各階段 `inn.cmd.gz` 指令歷史比對 `setAnalysisMode` 的使用時機：

階段	指令	意義
CTS (<code>03CCOPT</code> 起)	<code>setAnalysisMode -checkType setup / -checkType hold</code>	交替切換 <code>timeDesign</code> 要看哪一種違規
Route (<code>01Route</code> 起， CTS/Placement 都沒有)	<code>setAnalysisMode -analysisType onChipVariation</code>	首次開啟 OCV (On-Chip Variation) derate 分析
Route (<code>05MetalFill</code> 收尾)	<code>set locv_inter_clock_use_worst_derate false</code>	微調 LOCV (Location-based OCV) 跨時脈 derate 策略

重點：OCV 分析在 Placement / CTS 階段**沒有**打開，一路到 **Route** 才**第一次啟用**——實務上前段用單純 corner-based 分析先求快速收斂，等進入 routing、時序數字接近最終、才切換到較嚴格（也較耗時）的 OCV derate 分析做 sign-off 等級的檢查，呼應 Step 8 `05MetalFill` 最終驗證用的就是這套 OCV 設定下的時序結果。

Step 2 — Floorplan

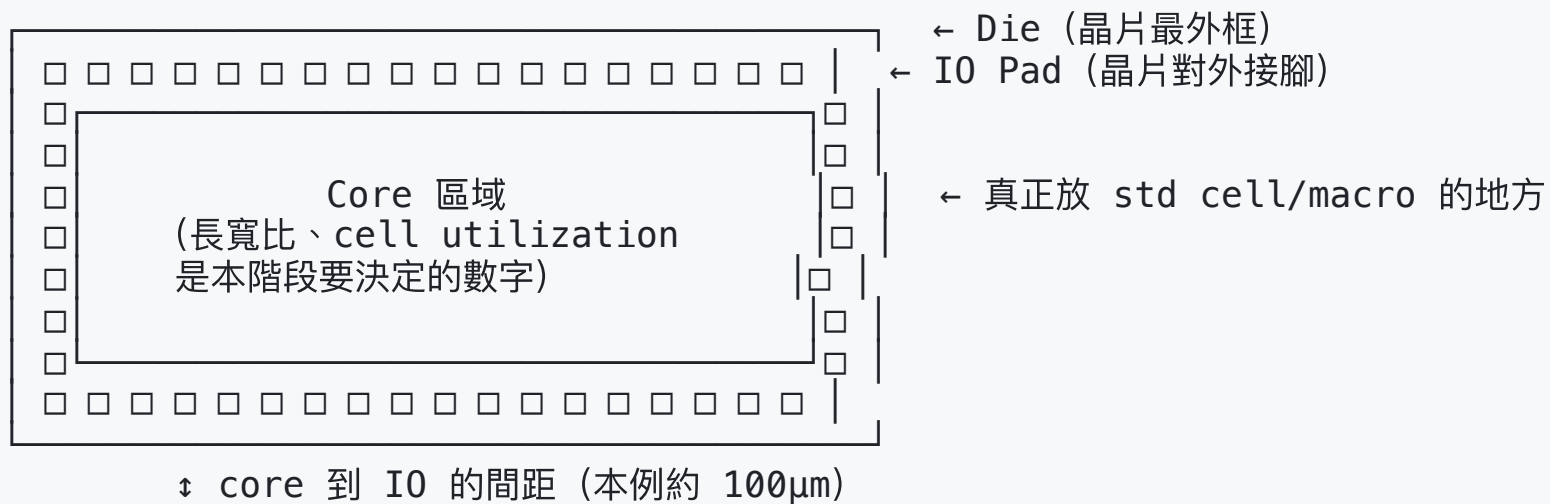
Step 2 : Floorplan 理論重點

這階段要做什麼：

- 決定晶片 **die / core** 尺寸（面積、長寬比）
- 決定 **cell utilization**（標準單元密度目標，通常 60–80%）
- 決定 core 到 IO 的邊界間距
- 評估巨集（macro）擺放位置（若有）
- 目標：面積最小、繞線總長最短、關鍵路徑延遲最小、routing 成功率最高

詳見 `note/floorplan.md` 3.1–3.2 節（ICC 用 `initialize_floorplan`）

Step 2：示意圖 — Floorplan 長什麼樣子



- **Die**：晶片實際的矽晶粒範圍
- **Core**：die 扣掉 IO 之後，真正拿來放邏輯電路的區域
- **Aspect ratio**：core 的長寬比 (1 = 正方形)
- **Cell utilization**：core 面積裡預計被 standard cell 填滿的比例 (通常 60–80%，留白給繞線與後續調整空間)

Step 2 : Floorplan 實作對照

gcd 範例 (90nm, 簡化) :

```
floorPlan -r 1 0.7 5 5 5 5
```

高寬比 1 (正方形)、cell utilization 0.7 (70%)、core 到四邊各 5 μ m。

DTMF_CHIP 實際案例 (180nm) :

```
floorPlan -site tsm3site -r 0.75 0.702385 100.94 100.44 100.32 100.24
```

高寬比 0.75、utilization $\approx$ 70.2%、core 到 IO 四邊約 100 μ m —— 學生先試算一次 (`-r 0.73 0.7 100 100 100 100`) 再定案，可見 floorplan 尺寸不是一次到位，通常會先抓大概數字、看後面 placement/routing 結果再調整。

Step 2 進階補充（可視時間彈性簡報）

Step 2 補充：什麼是 Partition ？

晶片太大、cell 數太多時，一次「扁平化 (flat)」placement/routing 會讓工具跑不動、收斂太慢，這時會把設計切成好幾個獨立區塊 (**partition**)，每個區塊各自做 placement/CTS/routing (像縮小版的 APR)，最後再組裝回頂層——這是**階層化 (hierarchical)** 設計，跟本簡報兩個案例用的扁平化流程相對。

頂層 (top)



← 每個區塊各自獨立做 placement/CTS/routing

- 好處：區塊之間可以平行開發、各自收斂，加速大型設計的疊代速度
- 代價：切分邊界要事先規劃好每個區塊對外露出哪些 pin，切壞了反而更難修

`gcd` / `DTMF_CHIP` 都用扁平化 placement (`setPlaceMode -fp false`，Step 4 已提過)，整個設計當一個區塊處理，沒有真的切 partition。

Step 2 補充：什麼是 Feedthrough ？

切成區塊後，區塊邊界變成固定的「牆」，訊號只能透過事先定義的 pin 進出。但若 A 區塊要送訊號給 C 區塊，實體佈局上卻剛好要「穿過」B 區塊——這時就在 B 加一個 **feedthrough pin**，訊號邏輯上跟 B 完全無關，只是被引導原封不動地穿過去繼續往 C 走。



feedthrough pin：訊號穿過 B，B 內部邏輯完全用不到這條線

指令歷史裡自然找不到 feedthrough 的實例——這是設計規模大到必須分區塊時才會遇到的問題，本次兩個案例規模都不需要切分。

Step 2 補充：同 Module 聚在一起 vs 階層化設計

這是兩個容易搞混、但完全不同層次的概念：

- **階層化設計 (Partition, 上一頁)**：整顆晶片實體上被切成好幾個獨立區塊，每個區塊有自己的 pin 邊界，各自做 placement / CTS / routing——是否要切，是設計規模的決定
- **同 Module 聚在一起 (hierarchical gravity)**：即使沒有切成區塊、整顆晶片仍是扁平化 (flat) placement，還是可以選擇要不要讓同一個 RTL module 的 cell 保持聚在一起——是單一 placement run 內部的擺放偏好

	階層化設計 (Partition)	同 Module 聚在一起
作用範圍	整顆晶片，切成獨立區塊	單一 flat placement 內部的擺放偏好
邊界	硬邊界，區塊間只能透過 pin 溝通	沒有實體邊界，只是傾向聚在一起
開啟時	各區塊可平行開發，加速大型設計	保留模組邊界完整性，方便除錯 / 可讀性
關閉時	不適用（不切分就是扁平化）	Cell 可自由混合擺放，更利於時序 / 壅塞最佳化

Step 2 補充：同 Module 聚在一起怎麼設定？

```
create_fp_placement -timing_driven -no_hierarchy_gravity    ;# 關閉：cell 自由混合，優先求最佳 QoR  
create_fp_placement -timing_driven                        ;# 開啟（VFP 預設）：保留模組邊界完整性
```

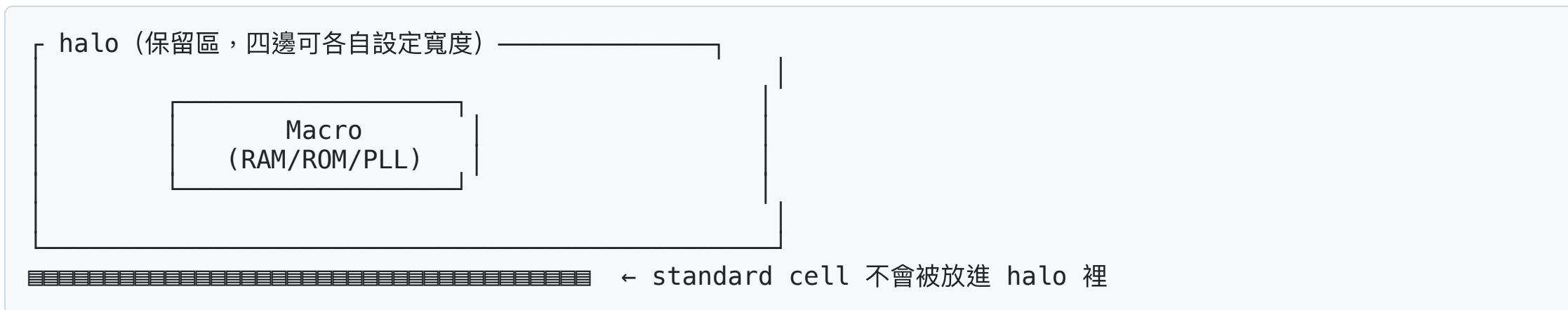
- **開啟（預設）**：模組的單元分布更規則，同一模組的邊界完整性保留下來，但不利於跨模組邊界的時序最佳化
- **關閉（`-no_hierarchy_gravity`）**：模組邊界不保證完整，但工具能更自由地把 cell 擺到對時序最有利的位址

`gcd` / `DTMF_CHIP` 兩個案例都是扁平化 placement，指令歷史裡也沒看到刻意保留 module 邊界的設定——小規模設計通常直接關掉 hierarchical gravity，讓工具自由最佳化，不用犧牲 QoR 去保模組完整性。

Step 2 補充：什麼是 Halo ？

(切完 partition / 決定好每個區塊的範圍後，區塊內部的巨集才會遇到下面這個問題)

巨集放好位置之後、加電源環之前，通常會先在巨集四周留一圈「保留區」——這就是 **halo** (也叫 keepout margin)。目的是避免 standard cell 貼著巨集邊界放，導致巨集接腳附近繞線塞爆，也預留空間給接下來要加的 power ring / stripe。



Step 2 補充：Halo 的真實案例

DTMF_CHIP 真實指令 (Version5, Floorplan/02MacroHaloset checkpoint) :

```
addHaloToBlock {15 15 15 15} DTMF_INST/ARB_INST/ROM_512x16_0_INST  
addHaloToBlock {25 35 15 25} DTMF_INST/PLLCLK_INST
```

{左 下 右 上} 四個數字是四邊的保留寬度 (μm) ——指令歷史裡這組數字被反覆調整了近 10 次，代表 halo 寬度跟 floorplan 尺寸一樣，是「先抓大概、看壅塞再調整」的疊代過程，不是一次定案。

對照 `note/floorplan.md` 3.2.12–3.2.15 節 (硬性 / 軟性 blockage、`set_keepout_margin`、`create_route_guide`)

Step 2 補充：Place Halo vs Routing Halo

Halo 概念上分兩種用途，只是不同工具的實作方式不太一樣：

	Place Halo（置放保留區）	Routing Halo（繞線保留區）
擋什麼	阻止 standard cell 被放進這個範圍	限制某些金屬層的訊號線不能繞過這個範圍
為什麼要擋	避免 cell 貼著巨集邊界，接腳附近繞線空間不夠	幫巨集自己的電源環/接腳留出走線空間，或避免數位訊號干擾類比巨集
ICC 對應指令	<code>set_keepout_margin -type hard/soft -outer {左 下 右 上}</code>	<code>create_route_guide -no_signal_layer {METAL5 METAL6} -coordinate {...}</code>
Innovus 對應指令	<code>addHaloToBlock {左 下 右 上} inst</code>	同一個指令一次設定，沒有再分開下第二道指令

DTMF_CHIP 真實案例只用了 `addHaloToBlock` 一種指令——代表 Innovus 把 place halo 跟 routing halo 合併成同一個保留區設定，不像 ICC 拆成 `set_keepout_margin`（置放） + `create_route_guide`（繞線）兩道指令；概念上仍是同一件事：**在巨集周圍留一圈「別人不能進來」的緩衝區**，保留給接下來的電源網路與訊號出線空間。

Step 3 — Power Planning

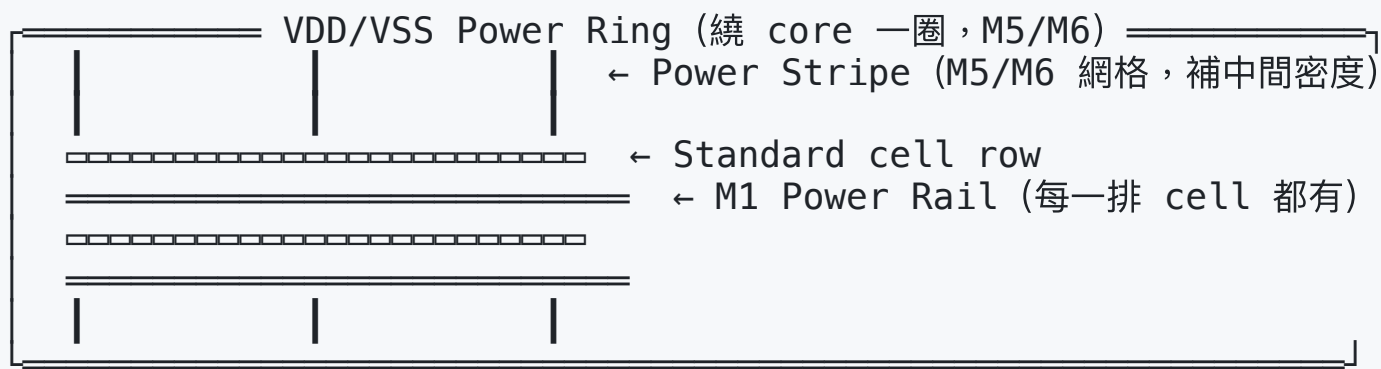
Step 3 : Power Planning 理論重點

這階段要做什麼：

1. 建立電源/地訊號的**邏輯連接**（把所有 instance 的 VDD/VSS pin 接上對應 net）
2. 在核心區周圍建立 **power ring**（VDD/VSS 電源環）
3. 建立 **power stripe**（電源網格，補足 ring 中間的供電密度）
4. 用 **sroute** 把電源網格一路連到標準單元的 **rail**（M1 電源軌）
5. 用 `verifyConnectivity` 確認電源網路無斷點

詳見 `note/floorplan.md` 3.5 節 PNS（Power Network Synthesis）；ICC 對應
`derive_pg_connection` + `create_pad_rings`

Step 3：示意圖 — 電源網路長什麼樣子



- **Ring**：繞著 core（與巨集）外圍的電源環，像主幹道
- **Stripe**：一整條一整條的電源網格，補足 ring 中間的供電密度，像次幹道
- **Rail**：每一排 standard cell 底下的 M1 電源軌，像街道，直接接到每顆 cell 的電源腳
- `sroute` 做的事：把 ring → stripe → rail 一路接起來，中間不能有斷點（否則 `verifyConnectivity` 會報錯）

Step 3 : Power Planning — gcd 範例

```
clearGlobalNets
globalNetConnect VDD -type pgpin -pin VDD -inst * -module {}
globalNetConnect VDD -type tiehi -pin VDD -inst * -module {}
globalNetConnect VSS -type pgpin -pin VSS -inst * -module {}
globalNetConnect VSS -type tielo -pin VSS -inst * -module {}

addRing -nets {VDD VSS} -type core_rings -follow core \
  -layer {bottom M5 top M5 right M4 left M4} -width 2 -spacing 0.5 -offset 0.1

sroute -connect { corePin } -layerChangeRange { M1 M5 } \
  -nets { VDD VSS } -corePinTarget { firstAfterRowEnd }
```

`globalNetConnect` 建立邏輯連接 → `addRing` 建 M4/M5 電源環 → `sroute` 把環一路接到標準單元列。

Step 3 : Power Planning — DTMF_CHIP 實際案例

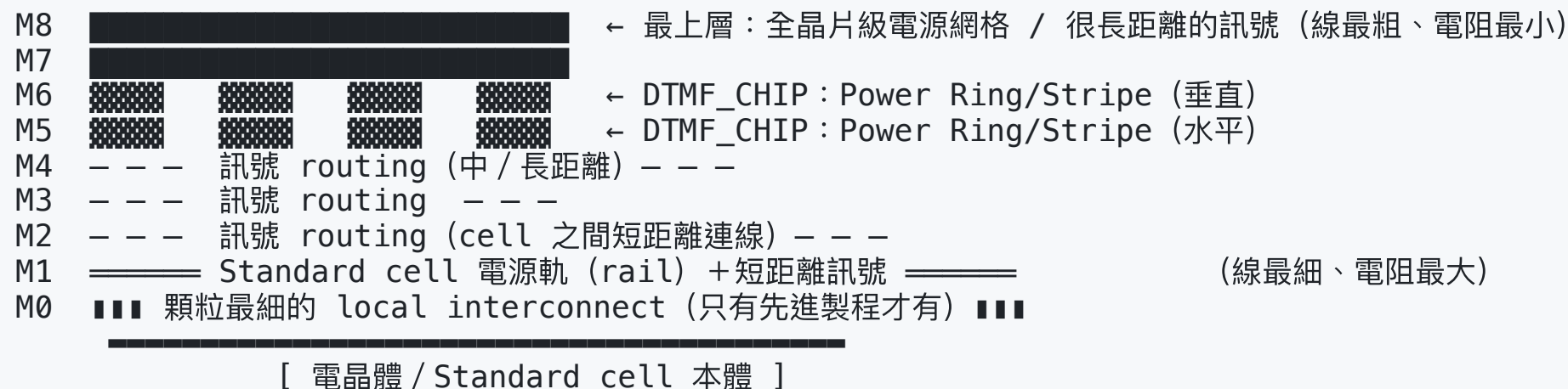
```
addRing -nets {VDD VSS} -type core_rings -follow core \  
  -layer {top Metal5 bottom Metal5 left Metal6 right Metal6} \  
  -width 7 -spacing 1 -offset 42.5  
addRing -nets {VDD VSS} -type block_rings -around selected ...    ;# 巨集周圍再加一圈  
  
addStripe -nets {VDD VSS} -layer Metal5 -direction horizontal \  
  -width 7 -spacing 1 -set_to_set_distance 264 ...  
addStripe -nets {VDD VSS} -layer Metal6 -direction vertical ...    ;# 多條不同 offset  
  
sroute -connect {blockPin padPin padRing corePin floatingStripe} \  
  -layerChangeRange {Metal1 Metal6} -nets {VDD VSS}  
verifyConnectivity -type all -error 1000 -warning 50    ;# 確認電源網路乾淨
```

真實案例除了 core ring，巨集也額外加了一圈 ring，並用多條 stripe 補密度，比 gcd 範例複雜得多。

Step 3 進階補充（可視時間彈性簡報）

Step 3 補充：金屬層 M0-M8 是什麼

前面的 ring / stripe / rail 其實都是選在不同的金屬層上做的。晶片內部是一層一層疊上去的金屬導線，由下往上大致長這樣（側視 / 剖面）：



- 由下往上：線越細、間距 (pitch) 越小 → 越適合精密的短距離連線，但電阻大；線越粗、間距越大 → 電阻小、能承受大電流，適合長距離走線與電源
- **M0**：只有較先進製程（通常 <45nm）才會有的「local interconnect」層，專門把電晶體接到 M1；180nm 這種較舊製程通常沒有 M0，電晶體直接由 M1 接出
- **M1**：幾乎所有製程共通的最下層通用走線層，standard cell 的電源 rail、同排 cell 間短距離訊

Step 3 補充：中高層金屬實際怎麼用

- **M2–M4（中層）**：cell 之間、跨 row 的訊號 routing 主力——DTMF_CHIP 最終總繞線長 320,640 μ m 全部落在 M1–M4（前一頁已展示的 DTMF_CHIP 實際案例）
- **M5–M6（次上層）**：線較粗、電阻較小，DTMF_CHIP 拿來做 Power Ring / Stripe；gcd 案例則是把 core ring 放在 M4/M5
- **M7–M8（最上層）**：層數更多的製程才會用到，通常給全晶片級電源網格或很長的 global 訊號

金屬層數是製程決定的，不是每個設計都用到全部：gcd 使用的 90nm 製程實際上支援到 M9（`addMetalFill -layer {M1...M9}`、via 可以跨到 M8），但因為電路太小，實際只用了 M1/M4/M5；DTMF_CHIP 的 180nm 製程整個流程最高只用到 M6——製程給的層數是上限，設計用多少層看複雜度與供電需求。

Step 3 補充：什麼樣的巨集需要 Block Power Ring

不是每個巨集都要另外加 ring，通常符合以下特徵才會加：

- **硬巨集 (hard macro)**：像 RAM / ROM / PLL 這種內部電路固定的 IP，內部沒有 standard cell 那種 rail 結構，需要一圈環把周邊電源接腳整合起來，才能穩定跟外部電源網路對接
- **耗電量大、電流密度高**：容量大的記憶體耗電相對集中，只靠 core 的 stripe 供電容易在巨集周圍造成 IR drop 過大，加一圈 block ring 能就近補強
- **獨立 / 類比電源域**：像 PLL 這種類比電路通常吃自己單獨一組 VDD/VSS（避免被數位開關雜訊干擾），需要專屬的 ring 而不是直接共用數位 core ring
- **離 core ring 較遠、位於晶片內部**：巨集若不是貼著 die 邊緣擺放，離主要電源環較遠，加 block ring 再往外接 stripe，比單靠核心 stripe 硬牽線更穩定

DTMF_CHIP 案例對照：從 MMMC 的 library set 就能看到這顆晶片有 `pllclk` (PLL)、`ram_128x16A`、`ram_256x16A`、`rom_512x16A` 幾個硬巨集——這正是前一頁 `addRing -type block_rings -around selected` 特別再包一圈的對象。

Step 3 : Power Planning 檢查清單

- [] 所有 instance 的電源腳都已邏輯連接 (`globalNetConnect` / `derive_pg_connection`)
- [] Core ring 已建立，巨集若有獨立供電需求也建立 block ring
- [] Stripe 密度是否足夠 (依 IR drop / EM 需求決定 stripe 數量與寬度)
- [] `sroute` 是否把 ring/stripe 一路接到 M1 標準單元 rail、macro pin
- [] `verifyConnectivity` 必須 0 error 才能進入下一階段 (floating power net 會讓後面所有時序分析失真)

Step 4 — Placement

Step 4 : Placement 理論重點

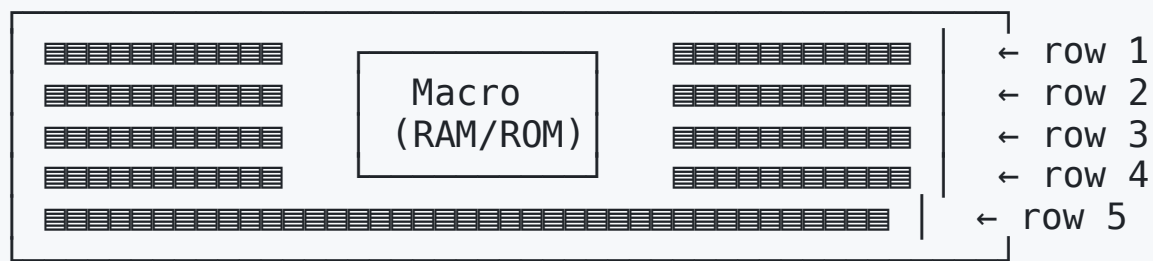
這階段要做什麼：

- 擺放巨集（若未在 floorplan 固定）與標準單元
- 時序驅動（timing-driven） + 壅塞驅動（congestion-driven） 雙重最佳化
- 處理 DFT：scan chain 的 `reorderScan`（依實體位置重新排列掃描鏈，減少繞線）
- Placement 前設定：忽略層、clock gating cell、tie cell 插入
- Placement 後跑 **pre-CTS optDesign**：先修 max cap / max transition，確認時序在 CTS 前就先收斂

詳見 `note/placement.md` 4.1、4.3、4.4 節

Step 4：示意圖 — Placement 長什麼樣子

Core 區域內部（俯視）：



每個 ■ 是一顆 standard cell (AND/OR/暫存器...), 一排一排貼著放

- **Macro**：像 RAM/ROM/PLL 這種面積大、形狀固定的巨集，通常先放好、擺放時不太挪動
- **Standard cell**：面積小、高度統一的邏輯閘/暫存器，一排一排（row）自動塞進剩下空間
- Placement 的目標：時序驅動 + 壅塞驅動雙重最佳化，讓需要互相連線的 cell 盡量靠近，減少後面繞線的難度

Step 4 : Placement — gcd 範例

```
setPlaceMode -fp false  
placeDesign -prePlaceOpt  
addTieHiLo -cell {TIELO TIEHI} -prefix LTIE  
setOptMode -fixCap true -fixTran true -fixFanoutLoad false  
optDesign -preCTS
```

`-fp false` = 扁平化（非階層化）placement；`placeDesign -prePlaceOpt` 一次做完「擺放＋前置邏輯最佳化」；`addTieHiLo` 插入固定高/低電位的 tie cell；`optDesign -preCTS` 做 CTS 前時序最佳化（先不修 fanout load 違例）。

Step 4 : Placement — DTMF_CHIP 實際案例

```
defIn scan_input_1.def
specifyScanChain scan1 -start IOPADS_INST/Pscanin1ip/C -stop IOPADS_INST/Pscanout1op/I
specifyScanChain scan2 -start IOPADS_INST/Pscanin2ip/C -stop IOPADS_INST/Pscanout2op/I

setPlaceMode -congEffort high -timingDriven 1 -clkGateAware 1 \
              -ignoreScan 1 -reorderScan 1 -placeIOPins 0
place_opt_design      ;# 反覆執行 5 輪才收斂
```

收斂結果（最後一輪 pre-CTS）	數值
Standard cell 數	5,561 顆 / Net 5,914 條
Cell utilization	≈76.5–77.5%
Pre-CTS setup WNS	全部為正值（0.04–0.15ns），0 違規路徑
DRC（ <code>verifyGeometry</code> ）	0 違規，乾淨

真實案例比教學範例多做的事：先匯入 scan chain DEF、指定兩條 scan chain 起訖點，並用 `place_opt_design` 反覆跑 5 輪才收斂 —— 教學範例只跑一次是因為電路太小、沒有真實收斂壓力。

Step 4 進階補充（可視時間彈性簡報）

Step 4 補充：什麼是 SAIF ？

Placement 階段如果要做**動態功耗最佳化**，需要知道每個訊號實際的**翻轉率 (toggle rate)** ——因為動態功耗 $\propto$ 電容 $\times$ 電壓² $\times$ 翻轉率，翻轉率估得不準，功耗分析就不準。

SAIF (Switching Activity Interchange Format) 就是紀錄翻轉率的檔案格式：讓 testbench 真正跑一段模擬向量，把每個訊號 0→1 / 1→0 的翻轉次數 (TC)、維持 0 / 1 的時間 (T0 / T1) 都記錄下來。

```
read_saif -input DESIGN.saif -instance_name I_TOP    ;# 讀入模擬得到的真實翻轉率
report_power                                           ;# 用真實翻轉率算動態功耗
```

沒有 SAIF 怎麼辦？ 只能手動用經驗值猜：

```
set_switching_activity -toggle_rate 0.02 a           ;# 針對個別 port 猜一個翻轉率
set power_default_toggle_rate 0.003                 ;# 其餘訊號用預設值
```

用猜的終究不如真實模擬資料準確——SAIF 的價值就在於「這是電路真的跑起來量到的數字」。

Step 4 補充：什麼是 UPF？

UPF (Unified Power Format) 是一組 TCL 指令構成的檔案格式，用來描述晶片的「**電源意圖 (power intent)**」：有哪些電源域 (power domain)、各自電壓多少、什麼情況下要關斷省電——這份意圖會驅動 EDA 工具在對的位置自動插入對的特殊 cell：

Cell	作用
Power Switch (header/footer)	實際切斷 / 接通某個電源域的供電
Isolation Cell	電源域斷電時把輸出鉗在固定值 (0 或 1)，避免下游浮接產生短路電流
Level Shifter	兩個不同電壓的電源域之間做電位轉換 (如 1.0V ↔ 1.2V)
Retention Register	斷電前先把暫存器資料存起來，復電後寫回去，不遺失狀態

為什麼在 Placement 處理：屬於某電源域的 cell 要放進該域的實體區域 (voltage area)，規劃得從 Floorplan 就開始準備。

`gcd` / `DTMF_CHIP` 都是單一電源域設計，找不到 UPF 指令——多電壓域 SoC 才會用到。

Step 4 補充：Power Switch Cell 怎麼運作？

UPF 提到的 Power Switch，實際上是插在「真實電源」與「虛擬電源」之間的一顆電晶體開關，用來實現**功率閘控（power gating）**——不用的模組直接斷電，消滅動態功耗、大幅降低漏電：

- **Header switch**：PMOS，接在真實 VDD 與電路的虛擬 VDD（VVDD）之間
- **Footer switch**：NMOS，接在真實 GND 與電路的虛擬 GND（VVSS）之間
- 開關本身要用**高 V_t** 電晶體——因為開關要長時間保持通 / 斷狀態，高 V_t 能降低開關自己的漏電流
- 實務上 **footer 用得比 header 多**：面積較小、驅動能力較強

Power Switch、Isolation Cell、Retention Register 這三種都屬於「**always-on**」邏輯——本身絕對不能被關斷電源，liberty 裡會標記 `always-on: true`，且通常有主 / 備兩組電源接腳確保一直有電。

Step 4 補充：精細粒度 vs 粗粒度功率閘控

	精細粒度 (Fine-grain)	粗粒度 (Coarse-grain)
開關位置	內建在 每一顆 standard cell 裡	獨立開關 cell，多顆邏輯閘共用一組
面積代價	增加 1x~3x	較省面積
優點	IR drop 控制更精細	對製程 / 電壓 / 溫度差異較不敏感，成本較低
難點	全域 sleep 控制訊號的驅動與 routing	要控制好開機瞬間的衝擊電流 (rush current) 與 IR drop
常見程度	較少見	業界較常採用

開關尺寸的取捨：開關 cell 越大，穩態時電壓降越小，但開機瞬間的 rush current 也越大——
`add_header_footer_cell_array` 可以把開關均勻分布在整個電壓域內，降低區域內的電壓降差異。

Step 4 補充：什麼是 Stress IR？

DFT 測試時，scan chain 會把測試向量一路「shift」進整顆晶片的暫存器——每個 clock cycle **幾乎所有暫存器同時翻轉**，這跟正常功能模式（只有部分邏輯依資料變化翻轉）完全不同：

- 正常功能模式：翻轉率通常只有幾 %～十幾 %，資料相依、分散在不同時間
- **Scan shift 模式**：翻轉率可以逼近 50%，而且**全部暫存器同一時間一起切換**

這種瞬間大量同時開關的電流需求暴增，會在電源網路上產生比功能模式**嚴重得多的**瞬時電壓降——這就是 **Stress IR (scan shift IR drop)**：不是功能上真的用得到這麼高的功耗，而是**測試模式本身「壓力測試」了電源網路**。

為什麼要在意：沒算過這個情境，晶片可能功能完全沒問題，卻在 ATPG at-speed test 時因為瞬間電壓降太大而**測試失敗**——問題不在設計，而在沒有針對 test mode 做電源分析。

Step 4 補充：怎麼減少 Stress IR？

- **降低 scan shift 頻率**：很多設計刻意把 shift clock 設得比功能模式慢，降低同時開關的瞬間電流
- **Scan chain 重排**：Step 4 已經看過的 `reorderScan`，除了減少繞線，也能降低相鄰暫存器同時翻轉的機率
- **多放一些 Decap**：呼應 Step 7 的 decap cell——在暫存器密集區多留去耦電容，緩衝瞬間電流需求
- **用 scan-shift 專用翻轉率跑 IR 分析**：這是上一頁 SAIF 概念的另一種用法——不是餵功能模式翻轉率，而是餵「shift 時全部暫存器同時翻轉」這種特殊翻轉率，模擬最壞情況的電源壓力

`gcd` / `DTMF_CHIP` 雖然都有匯入 scan chain (`specifyScanChain`)，但指令歷史裡沒有針對 scan shift 做專門的 IR 分析——這是量產晶片 DFT 簽核才會特別跑的一道分析。

Step 4 補充：checkpoint 陷阱

`01Placement.inn` 的存檔時機其實是剛設完 `setDesignMode`、`place_opt_design` 都還沒下的那一刻；真正的 5 輪 placement 是在同一個 Innovus session 裡繼續往下做、直到存下一階段的 `clk_tree.inn` 之前才發生。

checkpoint 檔名不代表「做完該步驟後」的狀態，要配合指令歷史（`inn.cmd.gz`）才能還原真實時間點——這個提醒之後在 Step 1 補充（附錄）比對 MMMC 設定時還會再用到同一招。

Step 4 : Placement 檢查清單

- ☐ 巨集位置是否合理（是否需要手動調整/固定）
- ☐ Scan chain 是否已 reorder（`reorderScan`），避免繞線暴長
- ☐ Congestion 是否可接受（壅塞熱圖無大面積熱點）
- ☐ Pre-CTS timing：setup WNS 是否為正（或至少可控）
- ☐ DRC（`verifyGeometry`）：cell / same-net / wiring 違規是否為 0
- ☐ 若時序/壅塞不理想 → 回頭調整 placement 策略或 floorplan，**不要硬撐進 CTS**

Step 5 — Clock Tree Synthesis (CTS)

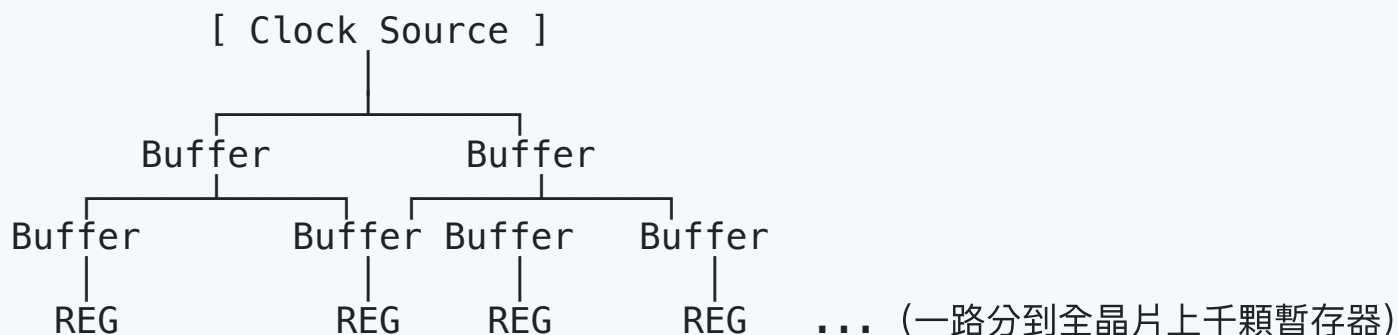
Step 5 : CTS 理論重點

這階段要做什麼：

1. 指定可用的 clock buffer / inverter cell (如 `CLKBUF*` 、 `CLKINV*`)
2. 產生並套用 clock tree spec (`create_ccopt_clock_tree_spec`)
3. 執行 `ccopt_design` : 插入緩衝器、平衡 skew、實際長出時脈樹
4. **post-CTS optDesign (setup)** : CTS 後修時序
5. **post-CTS optDesign -hold** : 時脈樹確定後，hold time 才能被精確計算與修復 (呼應 `note/STA.md` 「CT 插入後 hold 才能精確計算」)

詳見 `note/placement.md` 附錄 Step.5 (教材第 5 章尚未整理成獨立筆記)

Step 5：示意圖 — Clock Tree 長什麼樣子



- CTS 要做的事：從時脈源開始插入一層層 buffer，把時脈訊號「均勻地」送到每一顆暫存器
- 目標是讓每個暫存器收到時脈訊號的時間盡量一致 (**skew** 越小越好)，而不是像訊號線一樣抄最短路徑就好
- 時脈樹確定之後，**hold time** 才能被精確計算——這也是為什麼 hold 檢查要等 CTS 做完才有意義

Step 5 : CTS — gcd 範例

```
set_ccopt_property buffer_cells { CLKBUF* }  
set_ccopt_property use_inverters true  
create_ccopt_clock_tree_spec -file ccopt.spec  
source ccopt.spec  
ccopt_design -cts  
setOptMode -fixCap true -fixTran true -fixFanoutLoad true  
optDesign -postCTS  
optDesign -postCTS -hold
```

六個指令一次做完：指定 buffer → 產生規格 → 合成 → setup 最佳化 → hold 最佳化。

Step 5 : CTS — DTMF_CHIP 實際案例

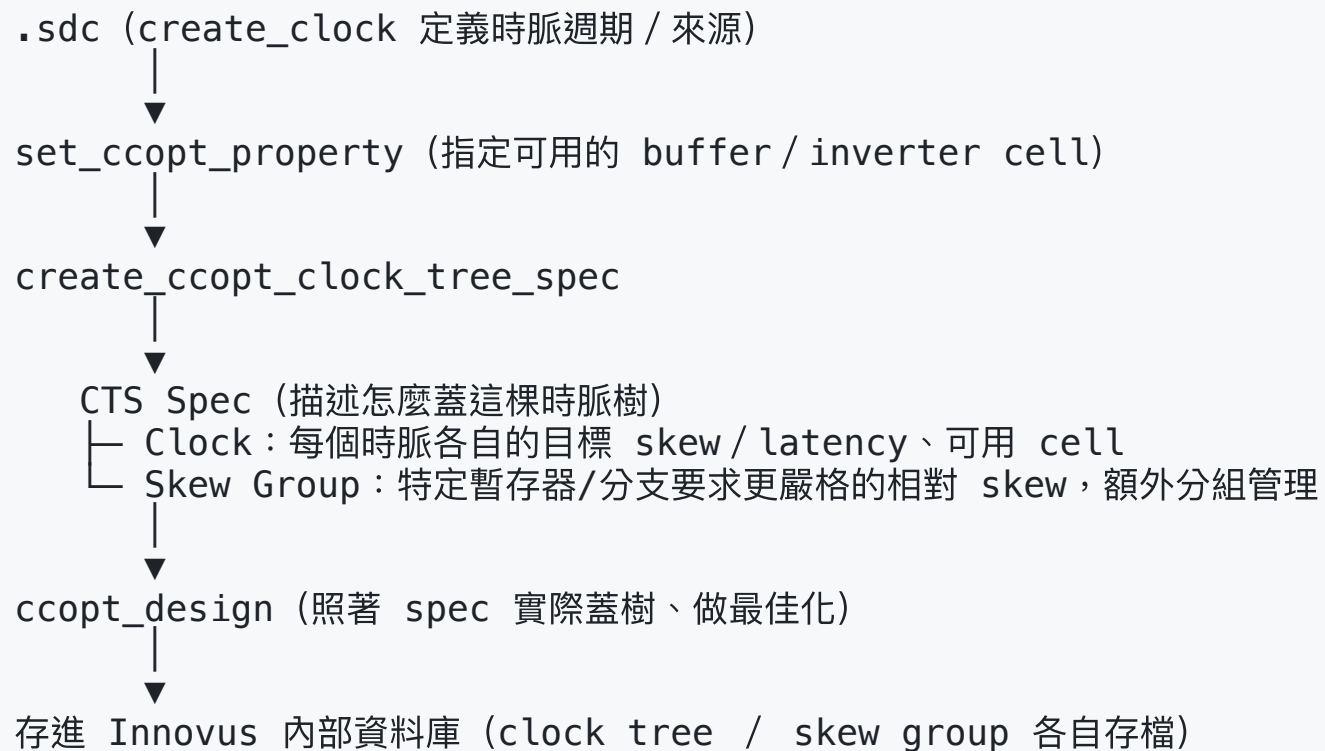
真實案例把 CTS 拆成好幾個 checkpoint，反覆調整：

```
clk_tree.inn      (placement 剛做完，CTS spec 還沒建)
↓
02CTS.inn         (建好 spec，尚未跑合成)
  set_ccopt_property buffer_cells {CLKBUF1 CLKBUF2 ... CLKBUFXL}
  set_ccopt_property inverter_cells {CLKINV1 ... CLKINVL}
  create_ccopt_clock_tree_spec
↓
03CCOPT.inn       (跑完合成：ccopt_design + 2 次 ccopt_design -cts)
↓
04ByitemCCOPT_1103.inn (兩天後回頭「逐項」再微調一次 ccopt_design -cts，
                        大量用 timeDesign -preCTS/-postCTS 匯出報告比對前後差異)
```

重點：真實案例的 CTS 不是跑一次就結束，而是先建 spec 存檔、跑完合成再存檔、事後還會回頭針對特定 clock domain 再跑一輪「by-item」微調——這是 CTS 常見的疊代模式。

Step 5 進階補充（可視時間彈性簡報）

Step 5 補充：從 .sdc 到 Clock Tree 的資料流



Step 5 補充：Spec 裡的 Clock 跟 Skew Group

- **.sdc**：只描述「這個時脈週期多長、從哪來」，不管實體怎麼實現
- **CTS Spec**：把 .sdc 的時脈意圖轉譯成「用什麼 cell、怎麼分組、允許多少 skew」的具體蓋樹規則
- **Skew Group**：不是所有暫存器都套用同一個 skew 目標——關聯緊密的一群暫存器（例如同一條資料通路）會被獨立分組，要求彼此之間的 skew 比全域目標更緊

Step 5 補充：Spec 跟 .db 是什麼關係？

`ccopt_design` 照著 spec 蓋完樹之後，**時脈樹結構**跟**每個 skew group 的分組結果**不是只存在記憶體裡算完就丟——會被存進 Innovus 內部的二進位資料庫，讓 checkpoint 存/取時不用重新計算。

真實證據：DTMF_CHIP 的 `CTS/02CTS.inn.dat/ccopt/` 資料夾裡，真的有 `clock_trees.bin` 跟 `skew_groups.bin` 兩個獨立檔案——證實 Innovus 把「時脈樹結構」跟「skew group 分組結果」分開存成內部資料庫檔案，`saveDesign` / `restoreDesign` 時直接讀寫這些檔案，不用每次都重跑一次 CTS 合成。

Step 5 補充：什麼是 DRV？

DRV (Design Rule Violation) 跟 Step 8 的幾何 **DRC** 不一樣——DRV 是電性上的違規，主要三種：

- **Max transition**：訊號從 0→1 / 1→0 切換得太慢（波形太「鬆軟」）
- **Max capacitance**：一顆 cell 驅動的負載電容超過它的驅動能力
- **Max fanout**：一顆 cell 驅動的下游接腳數量太多

為什麼 CTS 特別容易踩到：時脈源要一路分支、驅動全晶片上千顆暫存器，是全設計裡 fanout 最誇張的網路，電容 / fanout 特別容易爆表。

Step 5 補充：CTS 怎麼處理 DRV？

轉態變慢不只拖慢時序，還會讓 library 裡的時序數字失真（cell 本來只在 max transition 範圍內才有精確特性資料），連帶影響 skew / hold 判斷。

ccopt_design 除了平衡 skew，也同時在修 DRV：自動調整 buffer 尺寸、插入額外緩衝級，把每段 clock net 的轉態時間與負載壓回規格內。

DTMF_CHIP 真實證據：CTS 階段反覆呼叫 `timeDesign -postCTS -pathReports -drvReports -slackReports` —— `-drvReports` 就是專門把 DRV 違規（max cap / max tran / max fanout）另外拉出來一份報告，跟 `-slackReports`（時序）、`-pathReports`（路徑）分開看，是 CTS 收斂時例行要檢查的獨立項目。

Step 5 : CTS 檢查清單

- [] Buffer/inverter cell list 是否符合製程建議（避免用到不該用的高驅動力 cell）
- [] Clock tree spec 是否需要手動編修（真實設計很少直接用預設值）
- [] `cconst_design` 後檢查 **skew**、**latency**、**insertion delay** 是否在合理範圍
- [] Post-CTS setup timing：WNS 是否轉正
- [] Post-CTS **hold** timing：這是本階段最重要的新檢查項（CTS 前無法準確算 hold）
- [] 若特定 clock domain 仍有問題 → 針對該 domain 做「by-item」局部重跑，不用整個 CTS 重來

Step 6 — Routing

Step 6 : Routing 理論重點

這階段要做什麼：

1. 設定 router 模式（走線層上下限、疊代次數、時序驅動/SI 驅動開關）
2. **Global routing**：規劃網路的大致走線路徑與資源分配
3. **Detail routing**：實際產生每條線的幾何繞線
4. 修 DRC 違例（間距、寬度、短路、開路）
5. 修 **antenna 違例**（金屬走線過長累積電荷，需插 diode 或跳層修復）
6. Routing 完成後重新做 timing/DRC 檢查，不合格就繼續疊代

詳見 `note/routing.md` 6.1–6.2 節；NanoRoute（Innovus）對應 ICC 的 Zroute

Step 6 : 示意圖 — Global vs Detail Routing

Global Routing (先分格子，決定每條 net 大致要走哪個區域)

A	A \		B	B
A		A \	B	

↓

Detail Routing (在格子裡把每條線精確的金屬線路徑、via 都畫出來)



- **Global routing**：把晶片切成很多小格子（GRC），估算每條 net 大致要經過哪些格子、會不會塞車（congestion）
- **Detail routing**：在格子裡把每條線實際的金屬層、幾何座標、via 都畫出來，必須符合製程 DRC
- Metal layer 由下往上：M1 留給 std cell 電源軌與短距離訊號，越上層（本例 M5/M6）留給電源網路（呼應 Step 3），中間層（M2-M4）主要跑訊號——DTMF_CHIP 最終總繞線長

320,640 μm 全部落在 M1-M4

Step 6 進階補充：Congestion（可視時間彈性簡報）

Step 6 補充：Horizontal Congestion vs Vertical Congestion

Congestion 不是單一數字，而是分方向、分金屬層算的：

- 每個 GRC (Global Routing Cell) 有四個邊，量測這個邊「需要幾條走線 (demand)」vs「實際能提供幾條 routing track (supply)」，格式如 39/35
- 溢出 (overflow) = demand – supply，大於 0 代表這裡的線比軌道還多，繞不進去
- 每層金屬都有預設走線方向 (如 M1 水平、M2 垂直)，所以壅塞天生分成兩個方向：
 - Horizontal congestion (H)：左右方向的走線資源夠不夠
 - Vertical congestion (V)：上下方向的走線資源夠不夠

對照 `note/floorplan.md` 3.4.3 節；`report_congestion -grc_based -by_layer` 可分層看熱圖

Step 6 補充：真實壅塞報告怎麼看

```
phase5. Both Dirs: Overflow = 3621   GRCs = 2247 (2.73%)
phase5. H routing: Overflow = 1724   GRCs = 1139 (1.38%)
phase5. V routing: Overflow = 1897   GRCs = 1108 (1.34%)
phase5. METAL1   : Overflow = 1162   GRCs = 879  (2.13%)
phase5. METAL2   : Overflow = 1826   GRCs = 1079 (2.62%)
phase5. METAL5   : Overflow = 0       GRCs = 0    (0.00%)
phase5. METAL6   : Overflow = 0       GRCs = 0    (0.00%)
```

- 這份報告 H / V 溢出比例很接近 (1.38% vs 1.34%)，代表壅塞方向大致平衡；如果兩者差很多，通常代表 floorplan 長寬比或巨集擺放方向有問題（例如晶片被拉得很長很扁，某一個方向的走線通道天生較窄）
- **M5 / M6 Overflow 都是 0**——呼應 Step 3 補充的 M0–M8 概念：這兩層被劃給電源網路，訊號本來就很少走這裡，壅塞自然也低
- 經驗法則：任一 GRC 溢出大於 10，或超過 2% 的 GRC 邊沿有溢出，就算嚴重壅塞，該回頭調整 floorplan / placement 而非硬繞

Step 6 : Routing — gcd 範例

```
setNanoRouteMode -quiet -routeWithTimingDriven false  
setNanoRouteMode -quiet -routeWithSiDriven false  
routeDesign -globalDetail
```

教學範例關閉了時序驅動與訊號完整性（SI）驅動 routing 以求簡化、一次執行完 global+detail —— 實務設計通常會打開這兩項。

Step 6 : Routing — DTMF_CHIP 實際案例

```
routeDesign -globalDetail                ;# 01Route : 初版繞線
routeDesign -globalDetail -via0pt -wire0pt ;# 反覆疊代 via/wire 最佳化
                                           ;# (routeDesign 系列全流程共呼叫 103 次!)
                                           ;# 03Byitemopt_routing : 逐條 net 修違規
verifyConnectivity ...                   ;# (呼叫 74 次，穿插在每輪繞線之間檢查)
verifyProcessAntenna -report DTMF_CHIP.antenna.rpt -error 1000
                                           ;# (共呼叫 27 次，違規數逐漸收斂到 0)
```

routeDesign 103 次 + verifyConnectivity 74 次 + verifyProcessAntenna 27 次，三個數字加總說明 Routing 階段的本質就是「繞線 → 檢查 → 再繞線」不斷疊代，而非一次執行到位。

分支插曲：11/2 第一次繞完線後手動修 DRC 到 02Fix_done，但學生後來回頭多做一輪 CTS 微調 (Step 5 的 04ByitemCCOPT_1103)，重新繞線、直接跳過 02Fix_done，走向更乾淨的 03Byitemopt_routing → 04Antenna —— 代表 02Fix_done 是被放棄的舊嘗試，真實流程經常需要回頭重做前面階段，不是嚴格單向。

Step 6 進階補充：Routing 疑難雜症 (可視時間彈性簡報)

Step 6 補充：什麼是 SI / Crosstalk ？

SI (Signal Integrity, 訊號完整性) 關心的是「訊號波形有沒有被搞髒」，其中最常見的問題是串擾 (crosstalk)：兩條平行走線靠太近，中間有寄生耦合電容 / 電感，一條線的訊號變化會透過耦合「跳」到旁邊那條線：

- 靜態受害：旁邊那條線本來沒在動，卻被感應出一個毛刺 (glitch)，可能被誤判成邏輯訊號
- 動態受害：旁邊那條線剛好也在變化，訊號被加速或延遲，實際延遲跟算出來的不一樣

為什麼越來越重要：製程越微縮、線距越窄、單元密度越高，串擾影響越大——180nm 可以不太管，130nm 選做，**90nm 以下 sign-off 必須修**（呼應 Step 8 補充過的製程門檻）。

gcd 範例乾脆關掉 SI 驅動 routing (`-routeWithSiDriven false`) 求簡化；DTMF_CHIP 真實案例在不同輪次間開關切換 (`true` / `false` / `1` 都出現過)，代表實務上大多數時候會打開，只是某些疊代階段為了先求繞通而暫時關閉。

Step 6 補充：NDR 怎麼防串擾？

NDR (Non-Default Routing rule，非預設走線規則)：讓特定幾條網路不套用製程預設的走線寬度 / 間距，改用客製化規則——最常用在時脈訊號上：

- **加寬線寬**：降低走線電阻、減少 RC 延遲，讓 skew 更容易控制
- **加大線距**：拉開跟旁邊訊號線的距離，直接降低耦合電容，減少串擾

```
define_routing_rule my_ndr -spacings {...} -widths {...}  
set_clock_tree_options -routing_rule my_ndr      ;# CTS 階段套用到時脈網路
```

為什麼時脈網路優先套用：時脈是全晶片翻轉率最高、扇出最大、時序最敏感的訊號（呼應 Step 5 的 DRV 概念），最怕被鄰居串擾干擾到 skew，業界慣例是先把 NDR 規則套在時脈樹上，一般訊號線才維持預設規則。

Step 6 補充：SI / Crosstalk 還可以怎麼解？

NDR 主要用在時脈這種特別重要的訊號，一般訊號線的串擾則有更多元的修法：

- **Routing 時就開防範選項**：讓 router 在繞線階段主動避開容易串擾的走線方式，而不是等繞完再回頭修
- **加地線屏蔽（shielding）**：在兩條容易互相干擾的訊號線中間插一條接地線，直接隔開耦合
- **提高受害訊號的驅動能力**：upsized 受影響那條線的 driver，訊號本身抗干擾能力變強，比較不容易被鄰居牽動
- **少數修不掉的手動處理**：移動 driver 位置、換層繞線，拉開跟干擾源的距離

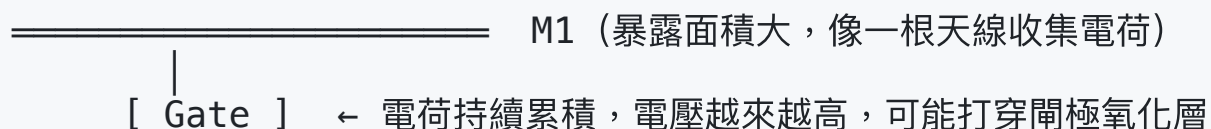
```
set_si_options -delta_delay true -route_xtalk_prevention true -static_noise true  
route_opt -xtalk_reduction -incremental ;# routing 完後針對串擾嚴重的地方最佳化
```

修復的兩大類：based-on-cell（換 cell、加驅動力）跟 based-on-routing（調整線距、換層）——先讓 router 自動修，少數自動修不掉的才手動介入。

Step 6 補充：Routing 怎麼解決 Antenna 違規（問題根源）

問題根源：晶圓廠用電漿蝕刻（plasma etching）把多餘金屬蝕刻掉來刻出線路。蝕刻過程中，一段還沒接到上層金屬 / 擴散區的長金屬線，會像天線一樣收集帶電離子，在它連接的電晶體閘極（gate）上累積電壓——電壓太高會把閘極氧化層打穿，電晶體永久損壞。

蝕刻進行中，金屬只接到低層、還沒接上層 = 危險狀態：

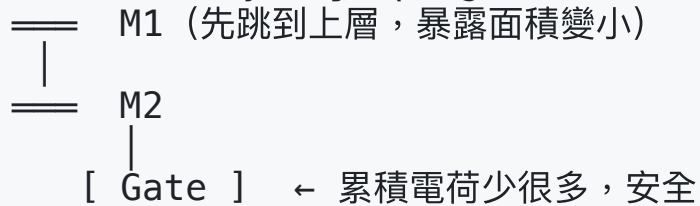


- **天線規則（antenna rule）：**晶圓廠規定「與某個 gate 相連的金屬總面積 ÷ 該 gate 面積」不能超過某個比值，超過就觸發 antenna violation
- Innovus 用 `verifyProcessAntenna` 檢查是否違反這個規則，DTMF_CHIP 案例被呼叫了 27 次，代表這是「繞線 → 檢查違規 → 再繞線」不斷疊代收斂的過程，不是一次到位

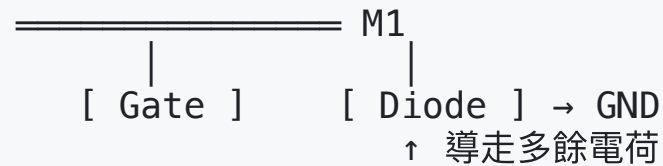
Step 6 補充：Routing 怎麼修 Antenna（跳層 / 插二極體）

兩種常見修法：

方法一：跳層 (layer jumping)



方法二：插二極體 (diode)



- **跳層**：讓 gate 連接的線儘早跳到上層金屬，蝕刻低層金屬時暴露面積變小，累積電荷自然變少——這是對既有繞線改動最小、router 最常用的做法
- **插二極體**：在 gate 連接的網路上接一顆反向二極體到 GND，把多餘電荷導走，但需要額外空間放二極體 cell

DTMF_CHIP 實際怎麼修的：指令歷史裡沒有看到手動插二極體的指令，也沒有特別開類似 `-insert_diodes_during_routing` 的選項——代表是 Innovus 的 NanoRoute 在 detail routing 時自動用跳層方式修，搭配 `-routeTopRoutingLayer 4`（訊號最高只走到 M4，呼應 Step 3 補充的金屬層概念）反覆 `routeDesign` 疊代，讓 27 次檢查裡的違規數一路收斂到 0。

Step 6：真實案例的 ECO 修正

postCTS 的 setup / hold 最佳化跑完後，指令歷史顯示學生發現 `SPI_INST` 底下好幾顆暫存器彼此距離太近、hold time 快不夠了，於是**手動下 ECO 指令**（而不是重跑整個 placement/CTS）：

```
ecoChangeCell -inst DTMF_INST/SPI_INST/spi_sr_reg_1 -downsize
ecoChangeCell -inst DTMF_INST/SPI_INST/dout_reg_1 -downsize
... （同樣手法共套用在 16 顆 SPI_INST 暫存器上）
```

- `-downsize` = 換成驅動力較弱、但延遲較大的同功能 cell，藉此**墊高 hold margin**
- 只動這 16 顆 cell，其餘已經繞好線、收斂好的部分完全不受影響——這就是最典型的 **ECO (Engineering Change Order)**：局部手術式修正，不是重跑整個階段

對照 Step 8 sign-off 結果：hold WNS 最終只有 -0.000ns、228 條路徑僅 1 條微幅違規——這種「幾乎壓線但過關」的結果，背後往往就有像這樣的手動 ECO 在支撐。

Step 6 補充：Hold 違規的「熱點」現象

真正除錯 hold 違規時，不會只看「總共幾條違規」，而是把每條違規路徑起點（launch 端 DFF 的 Q pin）畫到版圖上，看它們的實體 / 階層位置分布：

- **隨機散佈**：違規路徑東一條西一條，散布在整個晶片——通常代表**全域性**問題，例如 CTS skew 沒平衡好、OCV derating 設太保守
- **群聚成熱點 (hotspot)**：違規的 Q pin 集中在某一小塊區域或某個模組——通常代表**局部性**問題，例如那一區的 cell 間距太短、clock buffer 分支不平衡、或 IR drop 較嚴重

熱點比隨機違規好抓多了：知道問題「聚在哪裡」，就能針對那個區域看 clock latency / congestion / IR drop report，不用對整顆晶片大海撈針。

Step 6 補充：DTMF_CHIP 真實案例裡的熱點

回頭看前一頁那批 ECO：16 顆 hold 違規的 Q pin，全部集中在同一個階層模組 **DTMF_INST/SPI_INST** 裡，不是散落在整顆晶片——這就是一個典型的熱點。

代表問題不是 CTS skew 沒調好這種全域性狀況，而是 **SPI 模組本身的結構性問題**：shift register 一顆接一顆串接、reg-to-reg 距離很短，天生就容易踩到 hold。

這也是學生選擇用局部 ECO（downsize 那 16 顆）而不是重跑整個 CTS 的原因：熱點已經告訴他問題出在哪，對症下藥比全域重做更有效率。

Step 6 補充：如果晶片已經流片了，bug 怎麼修？

上一頁的 ECO 是流片（tapeout）前做的，電晶體位置都還沒定案，改起來相對自由。但如果晶片已經流片——電晶體已經透過光罩蝕刻在矽晶圓上，位置再也不能改變——發現 bug 總不能整套光罩重做（一套光罩要價數百萬到上千萬美金，還要再等好幾個月）。

Spare cell（空閒單元） 就是為了應對這種情況的保險：在 placement 階段，故意在設計各處多放一些沒接線、暫時沒作用的簡單邏輯閘（通常是 NAND2 / NOR2），平均分散在整個晶片裡。真的抓到 bug 時只要：

1. 把出問題的邏輯訊號斷開
2. 改幾條上層金屬線的 routing，把訊號接去附近的 spare cell，重新組出正確邏輯

→ 完全不用動下層電晶體光罩，只改幾層金屬光罩就能修，成本跟時間差好幾個量級。這種做法叫**光罩後 ECO（post-mask ECO）**，也稱 **freeze silicon ECO**。

Step 6 補充：Spare Cell 怎麼加、怎麼用

Placement 階段先埋好 spare cell (ICC 指令，Innovus 概念相同)：

```
insert_spare_cells -lib_cell {NAND2 NOR2} -num_instances 20 \  
  -cell_name SPARE_PREFIX_NAME -tie -hier_cell ALU  
  
set_dont_touch [all_spare_cells] true           ;# 沒接線也不會被當無用邏輯刪掉  
set_attribute [all_spare_cells] is_soft_fixed true ;# placement 不大搬動，CTS/routing 仍可微調
```

真的要修 bug 時：斷開錯誤邏輯的訊號 → 挑一顆離錯誤 cell 最近的 spare cell (減少金屬修改幅度) → 只改金屬層把訊號接過去 (route_zrt_eco)。

gcd / DTMF_CHIP 指令歷史裡都找不到 insert_spare_cells / spread_spare_cells，但 Innovus 的 setPlaceMode 其實已經內建 -ignoreSpare / -moduleAwareSpare 這兩個 spare cell 感知選項——工具早就準備好支援，只是這兩個練習案例沒有用到。

Step 6 : Routing 檢查清單

- ☐ Global routing 壅塞是否可接受（溢出 GRC 比例、最大溢出量）
- ☐ Detail routing 完成後 DRC 違規數是否收斂到 0
- ☐ Antenna 違規是否逐輪收斂到 0（`verifyProcessAntenna`）
- ☐ 繞線後 timing 是否還在可接受範圍（若嚴重劣化，代表 placement/CTS 需要回頭調整）
- ☐ 若单一分支修不乾淨 → 考慮回到 CTS 或 placement 重跑，而不是無限做 routing ECO 硬撐（見上頁真實案例）

Step 7 — DFM (Design For Manufacturing)

Step 7 : DFM 理論重點與實作對照

這階段要做什麼：

- **Filler cell**：填滿標準單元列間的空隙，維持 N/P 阱與電源軌連續
- **Metal fill (dummy metal)**：在各金屬層插入虛擬填充圖形，滿足晶圓廠金屬密度規則，並接到 VSS/VDD 避免浮空金屬

gcd 範例：

```
addFiller -cell FILL64 FILL32 FILL16 FILL8 FILL4 FILL2 FILL1 -prefix FILLER  
addMetalFill -layer { M1 M2 M3 M4 M5 M6 M7 M8 M9 } -nets { VSS VDD }
```

DTMF_CHIP 實際案例 (**05MetalFill** checkpoint，收尾階段)：

```
addFiller -prifix -doDRC      ;# 插 filler 同時做 DRC 檢查  
addMetalFill
```

詳見 `note/routing.md` 附錄 Step.7；ICC 概念對應 `insert_pad_filler` (`floorplan.md`

Step 7 進階補充（可視時間彈性簡報）

Step 7 補充：為什麼要放這些「非邏輯」cell？

除了做邏輯功能的 standard cell，APR 過程還會插入好幾種**沒有邏輯功能**、純粹為了製程 / 電性可靠度而加的特殊 cell：

Cell	放在哪裡	為什麼要放
Tie Hi/Lo	需要固定接高/低電位的接腳旁	有些輸入腳需要恆為 0 或 1（如未用到的功能腳），不能直接拉線接 VDD/VSS 了事，要透過 tie cell 才符合訊號完整性 / ESD 要求
Well Tap	每隔一段距離、standard cell row 中	定期把 N-well / substrate 接回 VDD / VSS，避免寄生 PNP 結構偏壓漂移觸發 latch-up （門鎖效應，會讓電路短路燒毀）
End Cap	每一排 cell row 的最左/最右端	保護 row 邊界的 well / 擴散區不被製程邊緣效應破壞，也避免邊界 DRC 違規（有時跟 well tap 合併成同一顆 cell）
Filler	標準單元列間的空隙	維持 N/P well 與電源軌連續，滿足製程對 cell density 的要求（上頁已介紹）
Decap	靠近耗電大 / 開關頻繁的邏輯附近	內建小電容補足局部瞬間電流需求，降低電源網路的 瞬時 IR drop / 雜訊

共同點：這五種 cell 都不做邏輯運算，純粹是為了讓電路能被穩定製造、穩定運作。

Step 7 補充：DTMF_CHIP 真實案例做了哪些？

-  **Filler**：05MetalFill checkpoint 明確下了 `addFiller -prifix -doDRC`
-  **一般電源腳連接**：`globalNetConnect VDD/VSS -type pgpin ...`
-  **Tie Hi/Lo**：指令歷史裡沒有 `-type tiehi/tielo` 或 `addTieHiLo`，代表這顆設計沒有需要固定接高/低電位的閒置腳，或這一步被跳過
-  **Well Tap / End Cap**：只看到 `setEndCapMode -boundary_tap false` ——把「end cap 順便當 well tap 用」的功能關閉，也找不到任何實際插入的指令
-  **Decap**：沒有找到任何 decap cell 插入指令

結論：這是一份課堂練習用的簡化流程，聚焦在 floorplan→placement→CTS→routing→sign-off 主線，沒有做到量產晶片會需要的 well tap / end cap / decap 收尾工作——理論上這五種 cell 都該懂，但看真實案例時要留意「學生範例 ≠ 量產完整流程」。

Step 7 補充：Endcap / Boundary Cell 的自動檢查與修復

除了手動下指令插入邊界 cell，EDA 工具通常有一道專門的**規則檢查 + 自動修復**流程，確保每一排 standard cell row 的邊界都符合規定：

1. **Check（規則檢查）**：掃過每一排 cell row 的起訖端、以及緊鄰巨集的邊界，確認每個該放 endcap / boundary cell 的位置都有放，沒有遺漏
2. **Autofix（自動修復）**：檢查到缺漏的地方，工具自動幫你插入正確的 boundary cell，不用逐一手動補

這種「先檢查、有問題自動補」的模式在 EDA 流程裡很常見——前面 Step 6 的 antenna 修復、這一頁的 filler 插入，邏輯上都是同一套：**檢查違規 → 自動或半自動修復**。

tCIC = TSMC 提供的 pre-checker（設計規則預檢工具，詳見 Step 8 補充）——這類代工廠輕量版 DRC 工具通常也涵蓋 endcap / boundary cell 規則，可以邊做邊跑。呼應前一頁：
DTMF_CHIP 只看到 `setEndCapMode -boundary_tap false`（關閉功能），沒有真的跑過這類檢查與修復。

Step 8 — Verification

Step 8 : Verification 理論重點與實作對照

這階段要做什麼（三項缺一不可）：

- **DRC** (`verifyGeometry`)：幾何設計規則檢查（間距、寬度、重疊）
- **LVS 概念** (`verifyConnectivity`)：連接性檢查（開路、短路），對應 IC 業界的 LVS
- **Antenna** (`verifyProcessAntenna`)：天線效應違例檢查

gcd 範例：

```
verifyGeometry  
verifyConnectivity -type all -error 1000 -warning 50  
verifyProcessAntenna -reportfile gcd.antenna.rpt -error 1000
```

Step 8 : DTMF_CHIP 最終 sign-off 結果

這裡的「sign-off」是什麼意思？前面各階段（Placement 反覆跑 5 輪、CTS 反覆微調）用的都是比較寬鬆、求快的分析設定；05MetalFill 這一版是用最嚴謹的 OCV 分析（見進階補充）重新驗收過的最終結果——通過 sign-off，才代表這批數字真的可信賴、可以交付。

DTMF_CHIP 最終（05MetalFill）sign-off 結果：

檢查項	結果
DRC（routeDesign.DRC.total）	0
verifyConnectivity	0 errors
verifyProcessAntenna	0 violations
Setup timing WNS(all)	0.022 ns（228 條路徑，0 違規）
Hold timing WNS(all)	-0.000 ns（228 條路徑中 1 條微幅違規）
總繞線長	320,640 μm（M1 23,049 / M2 97,520 / M3 120,783 / M4 79,288）
Via 總數	48,327（Via12 23,363 / Via23 19,244 / Via34 5,720）
Cell utilization（post-route）	66.8%

Step 8 進階補充（可視時間彈性簡報）

Step 8 補充：Sign-off 為什麼要換一套獨立工具？

Sign-off 嚴謹的不只是分析設定（OCV、corner），還包括換一套獨立的工具重新檢查一次，不是只信任 P&R 工具自己內建的檢查：

項目	平時疊代用（P&R 工具內建）	Sign-off 用的獨立工具
DRC	Innovus verifyGeometry	Hercules / Calibre 等專門的 DRC 工具
Timing (STA)	Innovus timeDesign	PrimeTime（業界公認的 sign-off 必選工具）
Crosstalk / SI	P&R 工具內建估算	Star-RCXT（精確 RC 抽取） + PrimeTime-SI

為什麼不能只信任同一套工具： P&R 工具的內建檢查是為了「一邊疊代一邊快速回饋」設計的，難免有簡化——例如 routing 階段的 DRC 用的是簡化的 FRAM view，不是完整電晶體級的 CEL view，可能遺漏問題。換一套獨立、專門做精確驗證的工具重新跑一次，才能抓到 P&R 工具自己可能漏掉的地方，這就像考卷不能自己改。

Step 8 補充：Sign-off 前還有一關——Foundry Pre-checker

實務上驗證不是只有「P&R 內建」跟「獨立 sign-off」兩級，中間通常還有代工廠提供的**輕量版預檢工具**：

P&R 工具內建檢查 (最快，較粗略) → Foundry Pre-checker (如 TSMC 的 tCIC) (較快，用代工廠真實規則跑一輪) → 完整 Sign-off (Hercules / Calibre) (最慢，最完整最準確)

- **為什麼需要中間這一關**：完整 sign-off DRC 跑一次可能要幾小時到幾天，設計還在疊代時不可能每改一次就跑一次；但只信任 P&R 工具自己的估算，又怕漏掉代工廠真正在意的規則
- **Pre-checker 的角色**：用代工廠提供的真實 design rule，跑一個比完整 sign-off 快很多的子集檢查，設計過程中可以常常跑，及早抓到問題（例如 endcap / boundary cell 有沒有正確插入），送完整 sign-off 時才不會一次冒出一堆違規

tCIC = TSMC 提供的 pre-checker，用來預先跑一輪設計規則檢查（包含 endcap / boundary cell 這類規則）並支援 autofix——就是這裡說的「中間那一關」，不用每次都跑完整、耗時的 Hercules / Calibre 才知道有沒有違規。

Step 8 補充：Sign-off 檢查項目會隨製程改變

Sign-off 該檢查什麼不是固定清單，會隨製程節點變重：

- **180nm**（本簡報 DTMF_CHIP 案例的製程）：可以不用特別考慮 crosstalk
- **130nm**：crosstalk 變成建議項目（可選）
- **90nm 以下**：crosstalk 是簽核階段必須修復的項目

製程越微縮、金屬線間距越窄、單元密度越高，串擾雜訊的影響就越大——這也是為什麼本簡報全程沒有出現 crosstalk sign-off 的討論：180nm 製程還不需要。對照 STA 三階段準確度遞增（合成後 < placement 後 < routing 後），**post-route STA 才是 timing sign-off 的依據**，因為那是唯一用上真實寄生參數與真實時脈樹的一次分析。

Step 8 補充：APR Report 為什麼要上傳到共用平台？

真實產線環境裡，光是「工具算出時序 / DRC 通過」還不夠——公司通常會把每一輪 APR / sign-off 的報告，統一上傳到一個**內部共用的資料平台**，讓不同團隊都能查閱、追蹤、比對：

- **設計團隊**：確認自己這顆 block 有沒有過關
- **整合 / 管理團隊**：一次看到所有 block 的 sign-off 狀態，掌握整體 tapeout 進度
- **跨版本比較**：晶片改版時，能跟上一版報告比對，看哪裡退步 / 進步

這類平台通常集中收 timing / DRC / power / QoR 各種報告，做成儀表板方便追蹤，而不是每個人各自散落一堆 log 檔案。

你提到的「Noob」（sign-off 工具）跟「Mind」（共用資料平台）這兩個名字，我沒辦法在公開資料或這個 repo 的筆記 / 參考設計裡找到對應確認，可能是課程或公司內部的特定命名——這裡先照你描述的**角色**（sign-off 工具、彙整報告的共用平台）說明概念，不代入確切工具名稱，避免給錯資訊。

Step 9 — Data Export

Step 9 : Data Export 理論重點與實作對照

這階段要做什麼：

- 設定分析模式（如 `bcwc`：Best-Case Worst-Case，同時看 setup 角與 hold 角）
- 輸出 **SPEF**（Standard Parasitic Exchange Format）：routing 後真實抽取的寄生電阻/電容，交給 sign-off STA 工具用
- 輸出 **SDF**（Standard Delay Format）：routing 後真實延遲，供 post-layout gate-level 模擬反標注
- 輸出 post-APR 網表、**GDSII** 版圖（送 tapeout）、**LEF abstract**（供上層當巨集用）、Innovus 資料庫存檔

對照 `note/STA.md`：`write_sdf` 輸出的 `.sdf` 會被 post-layout testbench 用 `$sdf_annotate` 反標注、跑 gate-level 模擬。

Step 9：匯出指令與實作對照

```
setAnalysisMode -analysisType bcwc  
rcOut -spef_file ${TOP_DESIGN}.spef  
write_sdf ... ${TOP_DESIGN}.sdf  
saveNetlist ${TOP_DESIGN}_apr.v  
streamOut ${TOP_DESIGN}.gds -mapFile ... -mode ALL  
write_lef_abstract ${TOP_DESIGN}.lef  
saveDesign ${TOP_DESIGN}.enc
```

註：DTMF_CHIP 180nm 練習案例的 checkpoint 紀錄止於 05MetalFill 收尾與驗證，並未包含 Step 9 匯出指令——這是課程練習到 sign-off 為止，尚未做最終 tapeout 匯出，此處以 gcd 教學範例呈現完整 Step 9 該做的事。

Step 9 進階補充（可視時間彈性簡報）

Step 9 補充：什麼是 SPEF ？

SPEF (Standard Parasitic Exchange Format) 記錄的是 routing 完成後，每一條金屬線實際量到的寄生電阻 / 電容——不是估算值，是真的把每段線的幾何形狀、跟旁邊線的耦合電容都抽出來的結果。

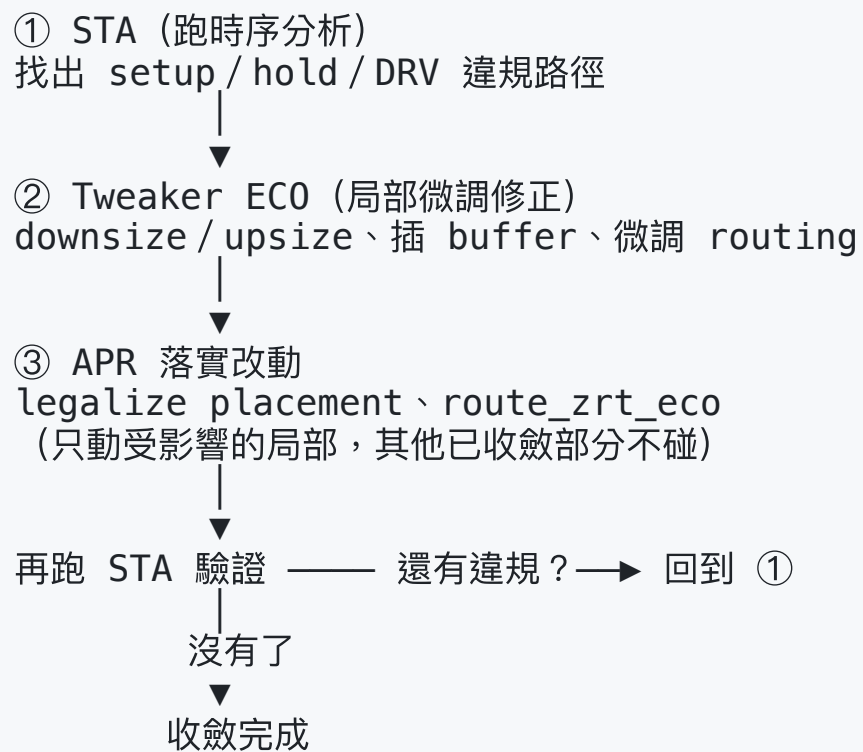
為什麼需要它：Step 5 提過 STA 準確度隨階段遞增（合成後統計估算 < placement 後估算走線 < routing 後真實寄生參數），**SPEF 就是「routing 後真實寄生參數」的具體檔案**——沒有它，STA 工具只能用估算值猜每條線的延遲；有了它，才能算出最接近實際晶片行為的時序數字。

跟 Step 8 sign-off 的關聯：Step 8 提過 sign-off 要換一套獨立工具（Star-RCXT + PrimeTime-SI）重新驗證——**Star-RCXT 做的事就是產生更精確版本的 SPEF**，再交給 PrimeTime 讀入做最終 timing sign-off，比 P&R 工具自己內建的寄生抽取更準確。

`rcOut -spef_file DESIGN.spef` 是 Innovus 輸出 SPEF 的指令；輸出後通常會搭配 `write_sdf` 一起用——SPEF 給獨立 STA 工具算時序，SDF 則是把算好的延遲數字包起來給模擬器用。

總結：STA ↔ ECO ↔ APR 的收斂循環

後端設計不是「做完就結束」，而是一直在跑這個循環，直到所有違規清零：



循環裡每一步在做什麼

- ① **STA**：檢查目前設計時序有沒有違規（Step 5 補充的 DRV、Step 8 的 sign-off 都是這一步）
- ② **Tweaker ECO**：不重跑整個階段，只對違規路徑做**最小幅度的**局部微調（downsize / upsize、插 buffer、微調 routing）——Step 6 的 `ecoChangeCell -downsize` 就是實例
- ③ **APR 落實改動**：把 ECO 的改動實際做進版圖，只 legalize / 重繞受影響的局部區域（`legalize_placement -eco`、`route_zrt_eco`），已經收斂的其他部分完全不受影響
- **回到 STA 驗證**：確認剛剛的修正真的解決問題，也沒有引入新的違規——還有問題就回到 ①，直到清零

為什麼這個循環貫穿整份簡報

本簡報看過的每一次疊代，本質上都是這個循環的實例：

- Placement 反覆跑 **5 輪**才收斂
- CTS 拆成好幾個 checkpoint，事後還回頭做 by-item 微調
- Routing 的 `routeDesign` 呼叫了 **103 次**、`verifyProcessAntenna` 呼叫 **27 次**
- SPI_INST 那批 **16 顆暫存器**的 downsize ECO，正是熱點分析後的 Tweaker ECO

核心心法：跑分析 → 找問題 → 局部修正 → 再驗證，不斷重複——這就是第一頁「RTL-to-GDS 九大步驟」結尾那句「做完 → 檢查 → 不合格就疊代重做」的具體樣貌。

動手練習：驗證你真的懂了

動手練習：如何 source checkpoint 深化理解

每個 `.inn` 存檔都封裝了完整的 lib/lef/mmmc 設定，可直接 `source` 還原到該階段狀態。與其死記指令，不如照下面順序**自己重新做一次**，比對能不能得到跟 checkpoint 一致的結果——做不到的地方，通常就是還沒真正理解該階段「為什麼」要這樣設定：

1. `source Floorplan/01Floorplan_set.inn` — 還原初始 floorplan，練習 `addRing` / `addStripe` / `sroute`
2. `source Placement/01Placement.inn` — 接著自己動手下 `place_opt_design`，比較是否能重現 Step 4 的收斂數據
3. `source CTS/02CTS.inn` — 練習下 `cchopt_design`，比對能否重現 `03CCOPT` 的 timing 數字
4. `source Route/01Route.inn` — 練習 `routeDesign -globalDetail -via0pt -wire0pt` 疊代收斂，最後跑 `verifyProcessAntenna` 與 `addFiller` / `addMetalFill` 收尾，比對能否重現 `05MetalFill` 的最終 QoR

每一步都建議先自己跑一輪、再對照 checkpoint 裡的既有數據，這樣才看得出「哪一輪疊代做了什麼調整」，而不只是複製指令。

各階段檢查清單總表

九大步驟一覽（要做什麼 + 完成判斷）

#	階段	要做什麼	完成判斷
1	Design Import	匯入 netlist/SDC/LEF/Lib/MMMC	無 load error
2	Floorplan	定 die/core 尺寸、utilization	面積、aspect ratio 合理
3	Power Planning	Ring/Stripe/Rail、 <code>sroute</code>	<code>verifyConnectivity</code> 0 error
4	Placement	擺放 + scan reorder + pre-CTS opt	Pre-CTS WNS 轉正、DRC 乾淨
5	CTS	建 spec、 <code>cchopt_design</code> 、post-CTS opt	Post-CTS setup/hold 收斂
6	Routing	Global+Detail routing、修 DRC/antenna	DRC = 0、antenna = 0
7	DFM	Filler、metal fill	密度規則通過
8	Verification	DRC / LVS / Antenna 三項齊全	全部 0 違規
9	Data Export	輸出 SDF/網表/GDS/LEF	交付檔案齊全

進階補充：EDA 工具與授權機制

進階補充：Cadence 的授權 (License) 機制

在真正下第一個指令之前，Innovus 其實要先跟**授權伺服器** (license server) 要到一張「使用權」，才能真的啟動：

- Cadence (以及大部分 EDA 廠商) 用的是 **feature-based** 授權：不是「買一套 Innovus 就什麼都能用」，而是每個功能模組 (Innovus 基礎版、CCOpt 時脈樹合成、Quantus 寄生抽取、CeltIC 訊號完整性…) 各自對應一張獨立的授權 **feature token**
- 啟動工具、或用到某個進階功能時，工具即時跟 license server**要 (checkout)** 一張對應 **token**；工具關掉或功能用完才還 (**release**)
- 環境變數 `CDS_LIC_FILE` (或 `LM_LICENSE_FILE`) 指向公司內部的授權伺服器位置

進階補充：為什麼授權機制值得在意

- 授權通常是**浮動式 (floating)** ——公司買固定數量的 token，多人共用，token 用完後別人就得排隊，不是每人都能同時跑
- EDA 授權**非常昂貴**（一張進階 P&R / 訊號完整性授權一年可能要價數萬到數十萬美金），公司會嚴格控管使用率，決定要不要加買
- 常見狀況：尖峰時段大家都在跑 CTS / routing，某個進階功能的 token 被別人占用，工具會卡住等待、甚至直接報錯拿不到授權——這不是設計錯誤，是資源排隊問題

`gcd` / `DTMF_CHIP` 的指令歷史裡看不到授權相關訊息（`inn.cmd.gz` 只記錄下過的 Tcl 指令，不含 license checkout 的系統訊息），但這是每次啟動 Innovus 都會在背景真實發生的一步。

參考資料

參考資料

理論筆記 (`note/`) :

- `floorplan.md` — Floorplan 與 PNS 電源網路理論 (含 gcd Step.1–3 對照)
- `placement.md` — Placement 與功耗控制理論 (含 gcd Step.4–5 對照)
- `routing.md` — Routing 與 ECO 理論 (含 gcd Step.6–9 對照)
- `STA.md` — 靜態時序分析理論 (含 SDF 反標注案例)
- `Version4_DTMF_CHIP_Innovus_flow.md` — DTMF_CHIP 完整實作紀錄

實作專案 (`reference_design/`) :

- `gcd/scripts/gcd_soc.tcl` — 90nm 教學範例，九步驟各跑一次
- `180um/Version4/` — 180nm 完整實作，checkpoint 資料庫 (`.inn` / `.inn.dat`)，含 MMMC / OCV 設定
- `180um/Version5/` — 同設計的另一次 Floorplan/Placement 練習，MMMC 設定與 Version4 完全一致，可互相驗證